Past Exams

- $w^* = argmin_{w \in R^D} ||y - Xw||_2^2$ is unique only if X is full rank
- Find minimum of function wrt a parameter θ by taking its gradient wrt θ, set it to 0
- Probability likelihood $\prod$; Log-likelihood $\sum$; $w^T w = ||w||_2^2$
- All linear/affine functions, sum of convex functions and composition of linear/affine functions and convex functions are convex.

Optimization

- Convex: $\mathcal{L}(\lambda \mathbf{w} + (1-\lambda)\mathbf{w}') \leq \lambda \mathcal{L}(\mathbf{w}) + (1-\lambda)\mathcal{L}(\mathbf{w}')\ \forall \lambda \in [0,1]$ or Hessian is PSD.
- Loss function should be symmetrical and penalize large and very large mistakes similarly (so outliers don't mess up the model)
- GD: $w^{(t+1)} = w^{(t)} - \gamma \nabla \mathcal{L}(w^{(t)})$ (γ step-size; for L-smooth convex $\mathcal{L}$, convergence for $0 < \gamma < 2/L$; linear models: $O(ND)$/iter).
- SGD: pick random n_t, $g_t = \nabla \mathcal{L}_{n_t}(w^{(t)})$, unbiased $\mathbb{E}[g_t] = \nabla \mathcal{L}(w^{(t)})$ ($O(D)$/step)
- Mini-batch: lower variance, parallelizable; $g_t = \frac{1}{|B|} \sum_{n \in B} \nabla \mathcal{L}_n(w^{(t)})$
- Momentum: $m_{t+1} = \beta m_t + (1-\beta) g_t$, $w_{t+1} = w_t - \gamma m_{t+1}$ (smooths noisy SGD, reduces oscillations, momentum = moving average of the gradient).
- Adam: $m_{t+1} = \beta_1 m_t + (1-\beta_1) g_t$, $v_{t+1} = \beta_2 v_t + (1-\beta_2) g_t \odot g_t$ (momentum + adaptive per-coordinate scaling; more stable steps; forgets older weights faster; scale-invariant to gradient). $\hat{m} = \frac{m}{1-\beta_1^{t+1}}$, $\hat{v} = \frac{v}{1-\beta_2^{t+1}}$; $w_{t+1} = w_t - \gamma\, \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$.
- SignSGD: $w_{t+1} = w_t - \gamma\, \text{sign}(g_t)$ (1 bit/coord; distributed sys; convergence issues)
- Subgradient g: $f(y) \geq f(x) + g^T(y-x)$, $\forall g$, any slope of a line that touches f at a point but stays entirely below f's curve. Used in nonsmooth convex GD as gradient
- Projected GD (constraints $w \in C$ convex): $w^{(t+1)} = P_C(w^{(t)} - \gamma \nabla \mathcal{L}(w^{(t)}))$ (brick wall $I_C(w) = 0$ if $w \in C$, ∞ otherwise; penalty method $argmin\ \mathcal{L}(w) + \gamma ||Aw - b||^2$).
- Feature standardization (pre-conditioning) helps ill-conditioned GD.
- GD iterations are cheaper than Newton's method, but require more to converge

Linear Regression

- MSE (Quadratic Loss): $L(w) = \frac{1}{2N} \|y - Xw\|_2^2 = \frac{1}{2N} \sum_{n=1}^N (y_n - x_n^T w)^2$.
- $\nabla L(w) = -\frac{1}{N} X^T (y - Xw)$. Hessian: $\nabla^2 L(w) = \frac{1}{N} X^T X$.
- Closed-form Least Squares (full col rank): $w^* = (X^T X)^{-1} X^T y$; if rank-deficient use regularization/pseudo-inverse np.linalg.solve (Gram $X^T X$: $O(ND^2)$, solve: $O(D^3)$).
- Linear Models are scale & shift invariant when there is no regularization

Overfitting & Underfitting

- Underfit (high bias): train err high & test err high (model too simple/bad features).
- Overfit (high variance): train err low but test err high (model too complex).
- Polynomial features: $\phi(x) = [1, x, x^2, \ldots, x^M]$, linear in ϕ but nonlinear in x.
- Regularization: $\min_w\ \mathcal{L}(w) + \Omega(w)$ controls complexity.
- Norms ($w \in \mathbb{R}^D$): $\|w\|_1 = \sum_{i=1}^D |w_i|$, $\|w\|_2 = \sqrt{\sum_{i=1}^D w_i^2}$, $\|w\|_\infty = \max_i |w_i|$.
- Ridge (ℓ_2): $\Omega(w) = \frac{\lambda}{2} \|w\|_2^2$; $w^* = (X^T X + N\lambda I)^{-1} X^T y$ (O like Least Squares)
- LASSO (ℓ_1): $\Omega(w) = \lambda \|w\|_1$ (no closed-form; promotes sparsity; optimum at corners), equivalent to imposing a Laplace prior on the weights, diamond-shaped
- Log-likelihood (Gaussian): $y_n = x_n^T w + \epsilon_n$, $\epsilon_n \sim \mathcal{N}(0, \sigma^2)$, $\log p(y \mid X, w) = log \prod_{n=1}^N p(y_n | x_n, w) = log \prod_{n=1}^N \mathcal{N}(y_n | x_n^T w, \sigma^2) = -\frac{1}{2\sigma^2} \sum_{n=1}^N (y_n - x_n^T w)^2 + c$
- Log-likelihood (Laplace): uses MAE instead, $p(y_n | x_n, w) = \frac{1}{2b} e^{-\frac{1}{b} |y_n - x_n^T w|}$.

Generalization, Model Selection, and Validation

- Generalization/True Error/Risk: $\mathcal{L}_\mathcal{D}(f) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(y, f(x))]$, the expected error over $\mathcal{D}$ true real-world distribution. Generalization gap: $|\mathcal{L}_\mathcal{D}(f) - \mathcal{L}_S(f)|$
- Empirical Error: $\mathcal{L}_S(f) = \frac{1}{|S|} \sum_{(x_n, y_n) \in S} \ell(y_n, f(x_n))$, unbiased estimator of the true error $\mathcal{L}_\mathcal{D}(f)$ (equivalent with enough data (Law of Large Numbers))
- Generalization Gap with test set: δ confidence threshold, $\mathbb{P}\left[\left| \mathcal{L}_\mathcal{D}(f) - \mathcal{L}_{S_{\text{test}}}(f) \right| \geq \sqrt{\frac{(b-a)^2 \ln(2/\delta)}{2|S_{\text{test}}|}} \right] \leq \delta$; gap $= \mathcal{O}(1/\sqrt{|S_{\text{test}}|})$.
- Probabilistic up. bound of test err $\mathbb{P}\left[\mathcal{L}_\mathcal{D}(f) \geq \mathcal{L}_{S_{\text{test}}}(f) + \sqrt{\frac{(b-a)^2 \ln(1/\delta)}{2|S_{\text{test}}|}} \right] \leq \delta$
- Hoeffding Inequality: $\mathbb{P}\left[\left| \frac{1}{N} \sum_{n=1}^N \Theta_n - \mathbb{E}[\Theta] \right| \geq \epsilon \right] \leq 2e^{-2N\epsilon^2/(b-a)^2}$, for i.i.d. $\Theta_n \in [a,b]$, $\Theta_i = l(y_i, f(x_i))$, empirical mean concentrated around the true one.
- K models (hyperparameter search): $\mathbb{P}\left[\max_{k \leq K} |\mathcal{L}_\mathcal{D}(f_k) - \mathcal{L}_{S_{\text{test}}}(f_k)| \geq \sqrt{\frac{(b-a)^2 \ln(2K/\delta)}{2|S_{\text{test}}|}} \right] \leq \delta$, can test many models without diverging a lot.

Classification

- 0-1 loss (binary): $\ell(y, f(x)) = \mathbf{1}_{y \neq f(x)}$; nonconvex/discontinuous.
- Bayes classifier $f^*(x) \in \arg\max_{y \in \{-1,1\}} \mathbb{P}(Y = y \mid X = x)$, minimize 0-1 loss in $\mathcal{D}$
- ERM (0-1): $\min_f\ \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{f(x_n) \neq y_n} = \min_g\ \frac{1}{N} \sum_{n=1}^N \mathbf{1}_{y_n g(x_n) \leq 0}$ with $f(x) = \text{sign}(g(x))$, g model output.
- Convex surrogate: replace $\mathbf{1}_{y_n g(x_n) \leq 0}$ by convex upper bound $\phi(y_n g(x_n))$ (e.g. hinge/logistic) $\Rightarrow$ convex ERM.
- Over-parametrization ($N < D$): if separable data, GD classification $\approx$ GD regression.

Logistic Regression

- Sigmoid $\sigma(\eta) = \frac{1}{1+e^{-\eta}} = \frac{e^\eta}{1+e^\eta}$, $\sigma'(\eta) = \sigma(\eta)(1-\sigma(\eta))$, $[-\infty, +\infty] \to [0,1]$ scale-invariant
- Log-odds/logit linear model: $\log \frac{p(y=1|x)}{1-p(y=1|x)} = \log \frac{p(y=1|x)}{p(y=0|x)} = x^T w + w_0$
- Predictions: $p(y = 1 \mid x) = \sigma(x^T w + w_0)$, $p(y = 0 \mid x) = 1 - \sigma(x^T w + w_0)$;
- MLE: Maximum Log-Likelihood, $w_* = \arg\max_w \prod_{n=1}^N p(z_n \mid w) = \arg\max_w \prod_{n=1}^N \sigma(x_n^T w)^{z_n} (1 - \sigma(x_n^T w))^{1-z_n}$, equivalent to Logistic Loss
- Logistic loss: negative log-likelihood, convex surrogate of the 0-1 loss, $w_* = \arg\min_w \sum_{n=1}^N -\log(p(z_n \mid w)) = \arg\min_w \frac{1}{N} \sum_{n=1}^N -y_n x_n^T w + log(1 + e^{x_n^T w})$
- Gradient: $\nabla L(w) = \frac{1}{N} \sum_{n=1}^N (\sigma(x_n^T w) - y_n) x_n = \frac{1}{N} X^T (\sigma(Xw) - y)$.
- Logistic loss penalizes deviation in the "wrong"direction much than the "correct"
- Linearly separable data: MLE is unbounded so $\|w\| \to \infty$ increases likelihood (fix with $\ell 2$ regularization). But it models a linear decision boundary
- No closed-form solution when minimizing negative log-likelihood
- SGD: updates gradient at all steps, $L(w) \neq 0\ \forall w$
- Newton's method: $w_{t+1} = w_t - \eta_t \nabla^2 L(w_t)^{-1} \nabla L(w_t)$ ($O(ND^2 + D^3)$/iter)

Support Vector Machines (SVM)

- Margin of hyperplane: $min_{n \leq N} |w^T x_n|$. Distance from hyperplane: $|w^T x| \ / \ ||w||_2$
- Max-Margin hyperplane: maximizes margin so if the training set slightly changes misclassifications will stay low $max_{w, ||w||=1} (min_{n \leq N} |w^T x_n|)$ $s.t.\ \forall n, y_n x_n^T w \geq 0$
- Hard-SVM: assume the dataset is linearly separable, $M = \frac{1}{||w||}$, $w' = \frac{1}{M} w$, $min_w \frac{1}{2} ||w||^2$ $s.t.\ \forall n, y_n x_n^T (w/M) \geq M \to y_n x_n^T w' \geq 1$
- Soft-SVM: allow constraint violations $min_{w, \xi} \frac{\lambda}{2} ||w||^2 + \frac{1}{N} \sum_{n=1}^N \xi_n$ $s.t.\ \forall n, y_n x_n^T w \geq 1 - \xi_n$, simplified to $min_w \frac{\lambda}{2} ||w||^2 + \frac{1}{N} \sum_{n=1}^N [1 - y_n x_n^T w]_+$ with $[a]_+ = max\{0, a\}$. Two terms balance regularization (variance) and accuracy (bias), w^* obtained with stochastic subgradient method
- Margin-based convex losses surrogate of 0-1 loss upper bounded by it ($\eta = y \mathbf{x}^\top \mathbf{w}$):
  - Quadratic Loss (MSE): $\text{MSE}(\eta) = (1-\eta)^2$, penalizes any deviation from 1
  - Logistic Loss: $\text{Logistic}(\eta) = \frac{\log(1+e^{-\eta})}{\log(2)}$, asymmetric cost; always penalizes.
  - Hinge Loss: $\text{Hinge}(\eta) = [1-\eta]_+$, penalizes low-confidence predictions.
- Convex duality: $G(w, \alpha)$ s.t. $\min_w L(w) = \min_w \max_\alpha G(w, \alpha)$ (primal) and $\max_\alpha \min_w G(w, \alpha)$ (dual). Always $\max_\alpha \min_w G \leq \min_w \max_\alpha G$; equality if G convex in w, concave in α. Dual often easier; α typically sparse (support vectors).
- SVM w/ duality: $[z]_+ = \max_{\alpha \in [0,1]} \alpha z$, $\min_w \max_{\alpha \in [0,1]^N} \left(\frac{1}{N} \sum_{n=1}^N \alpha_n (1 - y_n x_n^T w) + \frac{\lambda}{2} \|w\|^2 \right)$ convex in w and concave in α. Set $\nabla_w G = 0 \to w(\alpha) = \frac{1}{\lambda N} \sum_{n=1}^N \alpha_n y_n x_n = \frac{1}{\lambda N} X^T \text{diag}(y) \alpha$. So: $min_w L(w) = max_{\alpha \in [0,1]^n} \frac{1^T \alpha}{N} - \frac{1}{2\lambda N^2} \alpha^T \text{diag}(y) K \text{diag}(y) \alpha$, $K = XX^T$ (indep. of D; solve via QP/coord ascent).

kNN and curse of dimensionality

- kNN: local averaging method, based on the values of labelled neighbours, no training, just query $O(ND)$. *Small k:* $\to$ Low bias & High variance (complex boundaries)
- Curse of Dimensionality: high $d \to$ Training set covers less space and points are far apart, $P(x \in \text{box}) = r^d = \alpha$. If $\alpha = 0.01$: $d = 10 \to r = 0.63$; $d = 100 \to r = 0.95$
- Generalization Bound for 1-NN: Bayes classifier $f_*(x) = \mathbf{1}_{\eta(x) \geq 1/2}$ with $\eta(x) = p(1 \mid x)$, Bayes risk $L(f_*) = \mathbb{E}_X[\min\{\eta(X), 1-\eta(X)\}]$; assuming nearby points have the same label $\mathbb{E}_{S_{train}}[L(f_{S_{train}})] \leq 2L(f^*) + 4c\sqrt{d} N^{\frac{-1}{d+1}}$, last term is 0 if d constant and $n \to \infty$, increasing d and keep same err $\to$ increasing N exponentially
- Bound on Geometric Term: partition $\mathcal{X}$ into boxes $\{\text{Box}_k\}$, $p_k = \mathbb{P}(X \in \text{Box}_k)$. Then $\mathbb{P}(\exists x_i \in S_{\text{train}} : x_i \in \text{Box}_k \mid X \in \text{Box}_k) = 1 - (1-p_k)^N$; if box side length ε, worst-case within-box distance $\|X - x_i\| \leq \sqrt{D}\, \varepsilon$.

Kernel Ridge Regression and the Kernel Trick

- Alt. Ridge Regr.: $w_* = X^\top (XX^\top + N\lambda I_N)^{-1} y$; $O(N^3 + dN^2)$ vs $O(d^3 + Nd^2)$. Regularization term lifts eigenvalues by $N\lambda$, so we always can invert the whole term
- Representer theorem: for any loss l, minimizer of $\frac{1}{N} \sum_{n=1}^N l(x_n^T w, y_n) + \frac{\lambda}{2} \|w\|^2$ satisfies $w_* = X^T \alpha_*$ (optimal solution in $\text{span}\{x_1, \ldots, x_N\}$) with $\alpha_* \in \mathbb{R}^N$.
- Kernelized ridge (dual): $\alpha_* = \arg\min_\alpha\ \frac{1}{2} \alpha^T (\frac{1}{N} XX^T + \lambda I_N) \alpha - \frac{1}{N} \alpha^T y$
- Feature extractor: map $\mathbb{R}^d \to \mathbb{R}^{\hat{d}}$ (often $\hat{d} \gg d$) to make data linearly separable; explicit dot-products in $\hat{d}$ are expensive.
- Kernel trick: pick $\kappa(x, x') = \phi(x)^T \phi(x')$ to achieve the $\mathbb{R}^{\hat{d}}$ dot-products without computing ϕ, also measures similarity of points, save a lot on computation.
- Prediction: $y = \phi(x)^T w_* = \sum_{n=1}^N \kappa(x, x_n) \alpha_{*n}$ (linear in $\mathbb{R}^{\hat{d}}$, nonlinear in $\mathbb{R}^d$).
- Quadratic kernel (scalar x): $\kappa(x, x') = (xx')^2$; Feature map: $\phi(x) = x^2$
- Polynomial kernel: $\kappa(x, x') = (x^\top x')^2$ for $x, x' \in \mathbb{R}^3$; Feature map: $\phi(x) = [x_1^2, x_2^2, x_3^2, \sqrt{2}x_1x_2, \sqrt{2}x_1x_3, \sqrt{2}x_2x_3] \in \mathbb{R}^6$
- RBF kernel: $\kappa(x, x') = \exp(-\gamma \|x - x'\|_2^2)$ ($\gamma > 0$; implicitly infinite-dimensional ϕ).
- Build kernels: if κ_1, κ_2 kernels then $\alpha\kappa_1 + \beta\kappa_2$ ($\alpha, \beta \geq 0$) and $\kappa_1\kappa_2$ are kernels.
- Mercer: κ kernel iff symmetric and kernel matrix $K = (\kappa(x_i, x_j))_{i,j=1}^N \succeq 0\ \forall x_n$.

Neural Networks

- Double descent: test risk decreases as # params increases, spikes near interpolation/memorization, then decreases again when highly overparameterized $\to$ NNs can generalize (extra degrees of freedom *wiggle room* beyond fitting train points).
- NeuralNet: $f_{NN}(x) = f(x)^T w$ (learned representation $f(x)$ + linear prediction), non convex problem.
- L-layer MLP: $x^{(l)} = \phi(W^{(l)T} x^{(l-1)} + b^{(l)})$; output $h(x) = W^{(L+1)T} x^{(L)} + b^{(L+1)}$; params $\approx O(K^2 L)$. SGD weight update: $(w_{i,j}^{(l)})_{t+1} = (w_{i,j}^{(l)})_t - \gamma \frac{\partial \mathcal{L}_n}{\partial w_{i,j}^{(l)}}$
- Loss/output: regression $\frac{1}{2}(h(x) - y)^2$; binary $\log(1 + e^{-yh(x)})$ (logistic loss), $\hat{y} = \text{sign}(h)$; multiclass Cross-Entropy $-\log \frac{e^{h_y}}{\sum_i e^{h_i}}$.
- Activation functions: if nonlinear it enables nonlinear models, sigmoid σ; ReLU $[x]_+$ (applied element-wise); GELU $x\Phi(x) \approx x\sigma(1.702x)$ (differentiable ReLU).
- Barron UAT: if $\int_{\mathbb{R}^d} \|\omega\|\, |\hat{f}(\omega)|\, d\omega \leq C$, then $\exists\ f_n(x) = \sum_{j=1}^n c_j \phi(w_j^T x + b_j) + c_0$ s.t. $\int_{\|x\| \leq r} (f(x) - f_n(x))^2 dx \leq \frac{(2Cr)^2}{n}$; all smooth functions f can be approx. by a one-hidden-layer NN with error $O(1/n)$, high representational power. Sums of many simple (sigmoid/ReLU) units approximate complex functions (like Rielmann integrals); L_2 averages error (can hide peak errors), L_∞ controls worst-case errors.
- Piecewise-linear (PWL): any 1D PWL $q(x)$ can be written as affine + sum of shifted ReLUs, $q(x) = ax + b + \sum_{i=1}^m \alpha_i (x - \beta_i)_+$ (represented by 1-hidden-layer ReLU nets)
- Backprop: forward saves $x^{(l)}, z^{(l)}$. Error signal $\delta_j^{(l)} := \frac{\partial \mathcal{L}}{\partial z_j^{(l)}}$ with init $\delta^{(L+1)} = z^{(L+1)} - y$ and recursion $\delta^{(l)} = (W^{(l+1)} \delta^{(l+1)}) \odot \phi'(z^{(l)})$; gradients $\frac{\partial \mathcal{L}}{\partial w_{i,j}^{(l)}} = \delta_j^{(l)} x_i^{(l-1)}$, $\frac{\partial \mathcal{L}}{\partial b_j^{(l)}} = \delta_j^{(l)}$; compute all grads in $O(2K^2 L)$, forward in $O(K^2 L)$.
- Parameter Initialization: if init all weights and biases to 0 then all neurons in a layer act the same, chain rule multiplies many terms $\Rightarrow$ exploding/vanishing gradients; control layerwise variance (He init for ReLU): if width K, sample $w_i^{(l)} \sim \mathcal{N}(0, \sigma^2 I)$ with $\sigma^2 = 2/K$ so $\text{Var}[z^{(l)}] \approx 1$ across layers.
- BatchNorm: per feature (per neuron) normalize pre-activations in a mini-batch: $\bar{z}_n^{(l)} = \frac{z_n^{(l)} - \mu_B^{(l)}}{\sqrt{(\sigma_B^{(l)})^2 + \epsilon}}$, then $\hat{z}_n^{(l)} = \gamma^{(l)} \odot \bar{z}_n^{(l)} + \beta^{(l)}$ (learnable scale/shift so NN has the freedom to undo it); bias before BN is redundant (scale-invariant); regularization effect; inference uses fixed/running mean/var.
- LayerNorm: normalize per sample across features: $\bar{z}_n^{(l)} = \frac{z_n^{(l)} - \mu_n^{(l)} \mathbf{1}}{\sqrt{(\sigma_n^{(l)})^2 + \epsilon}}$, then $\hat{z}_n^{(l)} = \gamma^{(l)} \odot \bar{z}_n^{(l)} + \beta^{(l)}$; same for train and inference; common in transformers; quicker and more stable training.

Convolutional Nets

- CNNs: less params than Fully-Connected NNs and capture complete spatial structure; convolution (+ nonlinearity) + pooling blocks then Fully-Connected classifier.
- Convolution: $x_{n,m}^{(1)} = \sum_{k,l=0}^{K-1} f_{k,l}\, x_{nS+k, mS+l}^{(0)}$. Weight sharing (same filter at every position) + local connectivity $\Rightarrow$ translation equivariance. Hyperparams: kernel/filter size K, stride S, padding P; zero-padding helps borders and can preserve spatial size. Multi-channel conv: one filter per channel, sum across channels + bias.
- Pooling (max/avg): downsampling with hyperparams kernel/stride, no learnable ones
- Receptive field R_l grows with depth, early layers learn edges/colors, deeper ones learn objects. $W_{out} = \left\lfloor \frac{W_{in} + 2P - K}{S} \right\rfloor + 1$; $R_l = 1 + \sum_{j=1}^l \left((K_j - 1) \times \prod_{i=1}^{j-1} S_i \right)$. Region size in layer l to make a pixel of final output: $r_l = S^{(l)} \cdot (r_{l+1} - 1) + K^{(l)}$, $r_{L+1} = 1$
- Backprop with weight sharing: sum gradients over all edges that share a weight.
- Residual nets: use skip connections $Y = R(X) + X$ (residual branch R) to mitigate vanishing gradients (gradient can flow through identity path by setting $R \approx 0$).
- Weight decay (ℓ_2 reg): $min\ \mathcal{L} + \frac{\lambda}{2} \sum_l \|W^{(l)}\|_F^2$ (typically not on biases). GD update becomes $W = (1 - \eta\lambda) W - \eta \nabla \mathcal{L}$ (when the gradient vanishes weights go to 0 quickly); with BatchNorm, we get scale-invariance so we need other types of regularization.
- Dropout: during training keep each unit in layer l with prob. $p^{(l)}$ and train a random subnetwork; at test time use all units but scale activations by $p^{(l)}$ (or scale weights by $1/p^{(l)}$ during training). Regularizes but adds noise $\Rightarrow$ needs more iterations.

Ethics and Fairness in ML

- Avoid unfair discrimination and Stereotypes, understand Provenance of data.
- True error is an average criterion, ML models might achieve low error by minimizing for majorities and ignoring minorities.
- Removing a sensitive feature A may not remove its information if other features correlate with A.
- Fairness Criteria: score $R = f(X)$ where f is a model, outcome $D = 1_{R > t}$ with threshold t, A membership in protected group, Y real outcome; Independence: $R \perp A$ (equal acceptance rate); Separation: $R \perp A \mid Y$ (equal error rates, applied post-hoc); Sufficiency: $Y \perp A \mid R$ (calibration by group, A is irrelevant to predict Y using R).
- Calibration: $P(Y = 1 | R = r) = r$, by group if $P(Y = 1 | R = r, A = a) = r]$
- **Gaussian Distribution**: $f(x_i; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2} \right)$

Adversarial Machine Learning

- Data augmentation: make new training samples by transformation (regularization)
- Adversarial examples: small perturbations $\hat{x}$ causing confident misclassification.
- Adversarial risk: $R_\epsilon(f) = \mathbb{E}[\max_{\|\hat{x}-x\|\le\epsilon} \mathbf{1}_{f(\hat{x})\neq y}]$; use smooth surrogate $\max_{\|\hat{x}-x\|\le\epsilon} \ell(y\, g(\hat{x}))$ (g is the output of the NN before classification).
- White-box: taken wrt inputs (not the gradient, we change inputs to maximize loss) $\max_{\|\hat{x}-x\|\le\epsilon} \ell(y\, g(\hat{x})) = \ell(yg(x)) + max_{||\delta||\le\epsilon} \nabla_x \ell(yg(x))^T \delta$ (Linearization) with $\delta = \hat{x} - x$. Closed-form 1-step attacks: ℓ_∞ FGSM $\hat{x} = x + \epsilon\, \text{sign}(\nabla_x \ell(y\, g(x)))$; ℓ_2: $\hat{x} = x + \epsilon\, \nabla_x \ell / \|\nabla_x \ell\|_2$. Iterative attacks via PGD: $\delta^{t+1} = \Pi_{B_p(\epsilon)}(\delta^t + \alpha\, \nabla_x \ell(y\, g(x + \delta^t)))$ (for ℓ_∞ often use sign).
- Black-box: score-based (we know g) or decision-based (we know predicted class $f(x)$) query attacks; finite-difference grad approximation $\nabla g(x) \approx \sum_i \frac{g(x+\alpha e_i) - g(x)}{\alpha} e_i$.
- Transfer attacks: use surrogate model to prepare White-box attacks in Black-box scenarios; can become model stealing if we can query the real model.
- Adversarial examples we produce should not be like the ones models commonly encounter during training (e.g. different camera angles, phographic distortions)
- Adversarial training: $\min_\theta \frac{1}{N} \sum_n \max_{\|\hat{x}-x_n\|\le\epsilon} \ell(y_n\, g_\theta(\hat{x}))$ with inner maximizer $\hat{x}_n^*$ approximated by PGD; update θ with gradients on adversarial points $\frac{1}{N} \sum_n \nabla_\theta \ell(y_n\, g_\theta(\hat{x}_n^*))$. Robust and more interpretable gradients, but takes more PGD steps and has robustness/accuracy tradeoff (too large ϵ hurts clean accuracy). In comparison FGSM only takes an extra forward+backward pass.

Unsupervised Learning

- Learn from unlabeled data $\mathcal{D} = \{x_n\}_{n=1}^N$; representations (PCA/embeddings/matrix fact.), density/generative models (GMM/LLM), clustering/anomaly detection, EDA
- PCA is invariant to translation but not scaling, so apply stdscaling!
- Clustering: group similar objects, graph-based (partitioning), hierarchical (building a tree), model-based (mantain cluster models and infer membership K-means, GMMs).
- K-means: assign points to clusters with nearest mean, Within-Cluster Sum of Squares monotonically decreasing objective: $\min_{z,\mu} \sum_{n=1}^N \sum_{k=1}^K z_{n,k} \|x_n - \mu_k\|_2^2$, $z_{n,k} \in \{0,1\}$, $\sum_k z_{n,k} = 1$. Non-convex & NP-hard, algorithm assigns nearest centroid to all points then update $\mu_k = \frac{\sum_n z_{n,k} x_n}{\sum_n z_{n,k}}$; coordinate descent $O(NKd)$, finite steps convergence, no lower bound. Choose K via domain knowledge / elbow method. Fails on non-spherical/overlapping clusters and outliers (drag centroids away from dense areas). Matrix Factorization $\sum_{n=1}^N \sum_{k=1}^K z_{n,k} ||x_n - \mu_k||_2^2 = ||X^T - MZ^T||_{Frob}^2$
- K-means++: better init than random (avoids poor local minima, faster convergence, lower objective). Pick μ_1 from dataset; define $D(x_n) = \min_{j \le m} \|x_n - \mu_j\|_2^2$; sample next centroid with $\mathbb{P}(x_n) = \frac{D(x_n)}{\sum_i D(x_i)}$; repeat until K centroids, then run K-means.
- Prob. view: isotropic Gaussians $\Rightarrow$ maximizing likelihood $\Leftrightarrow$ minimizing WCSS.

Expectation Maximization (EM) Algorithm

- GMM: pick clusters $Z \sim \text{Mult}(\pi)$, then sample $x \mid (Z = k) \sim \mathcal{N}(\mu_k, \Sigma_k)$ (x member of cluster k), $p(x) = \sum_{k=1}^K \pi_k\, \mathcal{N}(x \mid \mu_k, \Sigma_k)$, params $\theta = \{(\pi_k, \mu_k, \Sigma_k)\}_{k=1}^K$
- GMM MLE: $\mathcal{L}(\theta) = \sum_{n=1}^N \log \sum_{k=1}^K \pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k) = \sum_{n=1}^N log(\sum_{k=1}^K p(z_n|\theta) p(x_n|z_n, \theta))$ (log-sum is hard; nonconvex; permutation of clusters gives same likelihood; can be unbounded). If labels were known $\rightarrow$ closed-form MLE (Hard-EM M-step): $\hat{\pi}_k = \frac{1}{N} \sum_n \mathbf{1}_{z_n = k}$, $\hat{\mu}_k = \frac{\sum_n \mathbf{1}_{z_n=k} x_n}{\sum_n \mathbf{1}_{z_n=k}}$, $\hat{\Sigma}_k = \frac{1}{\sum_n \mathbf{1}_{z_n=k}} \sum_n \mathbf{1}_{z_n=k} (x_n - \hat{\mu}_k)(x_n - \hat{\mu}_k)^T$.
- Hard-EM: assign hard labels $z_n = \arg\max_k \pi_k \mathcal{N}(x_n \mid \mu_k, \Sigma_k)$ then do MLE M-step
- K-means as special case of hard-EM: optimize only μ_k with fixed uniform weights $\pi_k = 1/K$ and fixed identical spherical covariances $\Sigma_k = \sigma^2 I$.
- Soft-EM: E-step: $q_{n,k} = p(z_n = k \mid x_n, \theta) = \frac{\pi_k \mathcal{N}(x_n|\mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(x_n|\mu_j, \Sigma_j)}$ (probabilities of belonging to each cluster, captures uncertainty). M-step: $\pi_k = \frac{1}{N} \sum_n q_{n,k}$, $\mu_k = \frac{\sum_n q_{n,k} x_n}{\sum_n q_{n,k}}$, $\Sigma_k = \frac{1}{\sum_n q_{n,k}} \sum_n q_{n,k} (x_n - \mu_k)(x_n - \mu_k)^T$. EM steps use params that depend on each other recursively $\rightarrow$ cannot use closed-form solution. Use coordinate (monotonic) ascent: E-step sets $q^{(t+1)}(Z) = argmax_q \mathcal{L}(q, \theta^{(t)}) = \arg\max_q \mathbb{E}_{Z\sim q}\left[\log\left(\frac{p(X,Z|\theta^{(t)})}{q(Z)}\right)\right] = p(Z \mid X, \theta^{(t)})$; M-step $\theta^{(t+1)} = argmax_\theta \mathcal{L}(q^{(t+1)}, \theta) = \arg\max_\theta \mathbb{E}_{Z\sim q^{(t+1)}}[\log p(X, Z \mid \theta)]$. $\mathcal{L}(\theta^{(t+1)}) \ge \mathcal{L}(\theta^{(t)})$, but can get stuck in local optima (not convex) and is not guaranteed to reach the global MLE. Clusters can overlap!
- EM lower bound: introduce any distribution $q(Z)$ (the labels we never had), incomplete likelihood: $\mathcal{L}(\theta) = \mathcal{L}(q, \theta) + \text{KL}(q \| p(Z \mid X, \theta))$ upper bound on $\mathcal{L}(q, \theta) = \mathbb{E}_{Z\sim q}[\log p(X, Z \mid \theta)] + H(q)$ (KL distance between $\mathcal{L}(\theta)$, $\mathcal{L}(q, \theta)$)
- Initialization: π_k uniform, μ_k K-means++, Σ_k from empirical feature σ^2, important!
- Pick K with cross-validation against validation set log-likelihood
- Anomaly detection: after fitting a GMM, score new points by likelihood $p(x)$

Alternating Least-Squares

- No-missing simplification: two steps coordinate descent, $Z^T = (W^T W + \lambda_z I)^{-1} W^T X$, $W^T = (Z^T Z + \lambda_w I)^{-1} Z^T X^T$ (ridge).

Bias-Variance Decomposition

- Data Model: $y = f(x) + \varepsilon$, $f(x)$ the true model, $\varepsilon \sim \mathcal{D}_\varepsilon$ (i.i.d. noise, $\mathbb{E}[\varepsilon] = 0$)
- Bias: $(f(x_0) - \mathbb{E}_S[f_S(x_0)])^2$, how far the expected prediction is from the true value $f(x_0)$. *Low model complexity $\rightarrow$ high bias, high complexity $\rightarrow$ low bias.*
- Variance: $\mathbb{E}_S\left[(f_S(x_0) - \mathbb{E}_S[f_S(x_0)])^2\right]$, measures the variability of predictions at x_0 across different training sets. *High model complexity $\rightarrow$ high variance.*
- Error Decomposition: at x_0 is $L(f_S) = \mathbb{E}_{\varepsilon\sim\mathcal{D}_\varepsilon}\left[(f(x_0) + \varepsilon - f_S(x_0))^2\right]$
- Bias-Variance Decomposition: $\mathbb{E}_{S,\varepsilon}\left[(f(x_0) + \varepsilon - f_S(x_0))^2\right] = \text{Var}[\varepsilon] + (f(x_0) - \mathbb{E}_S[f_S(x_0)])^2 + \mathbb{E}_S\left[(f_S(x_0) - \mathbb{E}_S[f_S(x_0)])^2\right] = Noise + Bias^2 + Variance$

Transformers

- Transformer: learned function that transforms input into contextualized representations by mixing information across tokens via self-attention. Text and images are flattened and mapped to real-valued vectors. Transformer block composed of (LayerNorm + SelfAtt) with a SkipConn and then (LayerNorm + MLP) with a SkipConn. $O(L(T^2K + TK^2))$ with L layers of width K and sequence length T
- Tokenization: split text in tokens with integer ID based on a vocabulary V
- BPE makes vocab. starting with a token per char, merging until desired vocab. size
- Token embeddings matrix $\in \mathbb{R}^{|V|\times D}$: map token ID to a $\mathbb{R}^d$ vector (input $X \in \mathbb{R}^{T\times D}$), mappings start random, learned via backprop during Transformer's training
- $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$, $\mathbb{R}^d \to [0,1]^d$, i dimension of $z \in \mathbb{R}^d$, scale-shift invariant
- Label smoothing: $\tilde{y} = (1-\varepsilon)y + \varepsilon\, \mathbf{1}/D$ (replaces hard one-hot targets with softer ones; reduces overconfidence & huge logits in softmax+CE, improves numerical stability).
- Self-attention: transforms contextualized representations using learned input-dependent weighted averages, $Q = XW_Q$, $K = XW_K$, $V = XW_V$; $P = \text{softmax}(QK^T/\sqrt{D_K})$ (row-wise, weighting coeffs $\in [0,1]$); output $Z = PV$. $p_{i,j}$ is based on how similar q_i, k_i are (dot product), normalize by $\sqrt{D_K}$ (stabilizes gradients for faster convergence). Positional embeddings W_{pos} learned via backprop are added to input as attention is order-agnostic (map position to feature vector $\{1, \dots, T\} \to \mathbb{R}^D$). Captures long-range deps within the context window. MLP layer applies same transformation to all tokens independently, $\text{MLP}(X) = \phi(XW_1 + b_1)W_2 + b_2$ (W_1, W_2, b_1, b_2 learned via backprop). $O(T^2)$ (parallelizable).
- Multi-head: many parallel attention patterns (like multiple channels in CNNs), compute Z_h per head, concatenate $Z = [Z_1, \dots, Z_H]W_O$, W_O learned via backprop
- Causal masks for decoders: sets $p_{i,j} = 0$ for $j > i$ (or add $-\infty$ above diagonal before softmax) to not predict based on following tokens.
- Used in Encoders (tagging (per-token classification), seq classification), Decoders (LLMs) and Encoder-decoders (seq2seq, translation). Everything can be tokenized!

LLMs & Text Representation Learning

- Co-occurrence (log) counts: $n_{ij} = \#\{\text{contexts with } w_i \text{ and } w_j\}$; sparse matrix $N = (n_{ij})$ (need vocabulary & context definition), perfect for Matrix Factorization.
- GloVe: weighted MF with $f_{dn} = \min\{1, (n_{dn}/n_{\max})^\alpha\}$, train via SGD or ALS.
- Skip-gram/word2vec: LogReg s eparates real (w_d, w_n) (window size ≈ 5) and fake (negative sampling, distinguish noise from data) maximize dot products for real pairs
- Words w_d with opposite meanings can have very similar context words w_n.
- Sentence/doc repr.: supervised (CNN/transformers/MF); fastText uses Bag of Words $x_n \in \mathbb{R}^{|\mathcal{V}|}$ with 2 layers (learned embeddings W + linear classifier Z), $min_{W,Z} \mathcal{L}(W, Z) = \sum_{s_n} f(y_n(WZ^T)\mathbf{x}_n)$ (not jointly convex). Unsupervised: averaging word vectors (baseline); train word vectors so averaging works (better); current: training with sentences as inputs to capture context (no single words, no BoW).
- Language models: self-supervised next-token prediction (massive general multi-task); multi-class (softmax over vocab.) or binary (real vs fake next word); factorization $p(x_{1:T}) = \prod_t p(x_t \mid x_{<t})$; Autoregressive inference (forward pass, pick next token by sample or argmax, append, repeat).
- Pretraining: general-purpose model from massive text; data sources (CommonCrawl, code, curated, synthetic); challenges: quality/diversity + trillion-token efficiency.
- GPT-1/2/3 (decoder-only), larger models improve (finetuning becomes optional, in-context learning), number of params and training set size increases.
- In-context learning: zero/one/few-shot task completion via examples in the prompt;
- Chain of Thought: showing examples of reasoning steps helps with complex tasks
- Posttraining: pretrained models may not follow user intent/policies; Supervised Finetuning is expensive, penalizes all mistakes equally and noisy (human labelling).
- Preference learning: collect pairwise comparisons (x_{prompt}, x_a, x_b), train reward model $R_\phi : X_{prompt} \times X_{response} \to \mathbb{R}$ with loss $\mathbb{E}[-\log \sigma(R_\phi(x_{prompt}, x_a) - R_\phi(x_{prompt}, x_b))]$.
- RLHF: finetune the policy π_θ (pretrained LLM) to maximize $J(\theta) = \mathbb{E}_{x_{prompt}\sim P_{data}, x\sim\phi_\theta}[R_\phi(x_{prompt}, x)]$; policy gradient $\nabla_\theta J = \mathbb{E}[R_\phi(x_{prompt}, x)\, \nabla_\theta \log \pi_\theta(x \mid x_{prompt})]$ (log-derivative trick, because we cannot take the derivative of the expected reward defined over the policy distribution ϕ_θ since it depends on the params θ we are optimizing).
- PPO: constrain/penalize updates to $\nabla_\theta J$ so that pretraining is not forgotten.
- Evaluation: loss/acc not comparable across tokenizers; use open-ended challenges + benchmarks (MMLU/ARC) + model/human arenas.

Matrix Factorization

- Low-rank model: $X \approx WZ^T$ with item factors $W \in \mathbb{R}^{D\times K}$, user factors $Z \in \mathbb{R}^{N\times K}$.
- Objective: $\min_{W,Z} \frac{1}{2} \sum_{(d,n)\in\Omega} [x_{dn} - (WZ^T)_{dn}]^2 + \frac{\lambda_w}{2} \|W\|_F^2 + \frac{\lambda_z}{2} \|Z\|_F^2$.
- Not jointly convex / not identifiable; choose K (too large $\Rightarrow$ overfit).
- SGD on one observed (d, n): with error $e_{dn} = x_{dn} - w_d^T z_n$, update uses $\nabla_{w_d} = -e_{dn} z_n$, $\nabla_{z_n} = -e_{dn} w_d$ (+ reg.).

Self-Supervised Learning

- Transfer learning: finetune pretrained model on related task with few labels (which are expensive), good performance on the related task.
- Self-supervised: create labels from input x via $g : x \mapsto (x_{in}, x_{out})$, learn pretext task $f : x_{in} \to x_{out}$ (e.g. text = MLM / next-token; vision = rotation, colorization).
- Masked Language Modelling (MLM): mask tokens (`[MASK]`) and predict them (learn contextual representations for downstream tasks).
- BERT (encoder-only bidirectional transformers): input = 1–2 sentences with `[CLS]` and `[SEP]`; encoder attends to full sequence. Masking: replace $\approx$ 15% tokens with `[MASK]` (sometimes random token / unchanged to reduce train-test shift for downstream tasks that don't mask). Embeddings: token + positional + segment/sentence membership embeddings. Predict original tokens for masked positions in parallel independently (softmax cross-entropy); NSP via `[CLS]` predicts if sentence B follows A. Can be finetuned for sequence classification and sentiment analysis via `[CLS]`, token classification (NER) via per-token outputs, retrieval tasks.
- Joint embedding Methods: encoder invariant to some transformations of data, two augmented views $(x_1, x_2) = g(x)$, learn encoder $f : X \to \mathbb{R}^d$ s.t. $f(x_1) \approx f(x_2)$; avoid collapse (f always predicting the same constant) via contrastive or regularized non-contrastive (e.g. BYOL w/ exponential moving average target encoder).
- Contrastive Learning: push away representations of unrelated views from different data points, x and a view of it x^+, any negative view x'^- we want $s(f(x), f(x^+)) > s(f(x), f(x^-))$ where s is the similarity of two embeddings
- SimCLR: sample two augmented views (x, x^+) of same example + many negatives $\{x_i^-\}$; optimize InfoNCE $L(x) = -\log \frac{e^{s(f(x), f(x^+))}}{e^{s(f(x), f(x^+))} + \sum_{i=1}^N e^{s(f(x), f(x_i^-))}}$. encoder f_1 + projector f_2 (train both end-to-end; use f_1 downstream); cosine similarity w/ temperature τ: $s(e_1, e_2) = \frac{\langle e_1, e_2\rangle}{\|e_1\|_2 \|e_2\|_2\, \tau}$; strong data augmentations.
- CLIP: contrastive image–text; maximize cosine similarity of matched (image, caption) pairs, minimize mismatched pairs; capable of few-shot learning+zero-shot prompting.

Generative ML

- Discriminative models $p(y \mid x)$; Generative models model $p(x)$ (or $p(x \mid y)$)
- Explicit density models define a density/likelihood; Implicit density models generate realistic samples but can suffer from mode collapse (low diversity, e.g. GANs)
- Tractable density models: explicit, exact likelihood, Autoregressive (LLMs, slow sampling) or Normalizing Flows (efficient sampling/inference via invertible transformations, computationally intesive)
- Approximate models: explicit, VAE (optimize lower bound on likelihood, stable but blurrier samples), Energy-based or Diffusion (high quality audio/images but slow).
- GAN: generative adversarial network, generator $G(z) \in p_g$ with $z \sim p_z$ (noise) and discriminator $D(x) \in (0,1)$ with $x \sim p_d$; differentiable two-player min-max game $\min_G \max_D \mathbb{E}_{x\sim p_d}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))]$; first term is how good is D at recognizing real samples (can't be used for classification); second term is G wanting to fool D as much as possible while D does the opposite; optimum when $p_g = p_d$. Trained with Adam by Alternating-GAN (sample x, z, maximize D, sample z again, minimize G, repeat). Conditional if conditions G, D on extra info (image, text or one-hot encod. class labels).
- Deep Convolutional GAN: for images, deconvolution layers that do image upsampling
- Diffusion models: progressively add noise via forward Markov chain $p(x_{0:T}) = p_0(x_0) \prod_{t=0}^{T-1} p(x_{t+1} \mid x_t)$, then generate samples by reversing it: $p(x_{0:T}) = p_T(x_T) \prod_{t=0}^{T-1} p(x_t \mid x_{t+1})$ (backward transition, iterative denoising).
- Ancestral sampling (score-based): start $X_T \sim p_{\text{ref}} \approx \mathcal{N}(0, \mathbb{I})$ and for $t = T-1, \dots, 0$ sample $X_t \sim p(\cdot \mid X_{t+1})$. With Gaussian forward step $p(x_{t+1} \mid x_t) = \mathcal{N}(\alpha x_t, (1-\alpha^2)\mathbb{I})$, reverse is approximated w/ Taylor expansion of $log\ p(x_t)$ around x_{t+1} (because we have no way of knowing $p(x_t|x_{t+1})$ when going backward), $p(x_t|x_{t+1}) = p(x_{t+1}|x_t) exp[log\ p(x_t) - log\ p(x_{t+1})] \approx \mathcal{N}(x_t; (2-\alpha)x_{t+1} + (1-\alpha^2)\nabla log\ p(x_{t+1}), (1-\alpha^2)\mathbb{I}_d)$ where $\nabla log\ p(x_{t+1})$ is the score (intractable as it requires calculating integral over every possible clean image x_0). Approximate it using a NN $s_\theta(t, x)$ (Denoising Score Matching) conditioned on X_0 with loss $\mathcal{L}(\theta) = \frac{1}{2}\mathbb{E}_{X_t}[||\nabla log\ p(X_t) - s_\theta(X_t)||^2] = C_2 \frac{1}{2} \mathbb{E}_{X_0} \mathbb{E}_{X_t|X_0}[||s_\theta(X_t) - \nabla log\ p(X_t|X_0)||^2]$; predict directly noise added $\hat{\xi}_\theta(t, X_t)$ with $s_\theta(t, X_t) = -\hat{\xi}_\theta(t, X_t)/\sqrt{1-\alpha^{2t}}$ ($\mathcal{L}(\theta) = \frac{1}{2} \sum_{t=1}^T \mathbb{E}_{X_0} \mathbb{E}_{X_t|X_0}[||\hat{\xi}_\theta(t, X_t) - \xi_t||^2]$). Train DSM with forward transitions updating θ with SGD computed at a random t and $\xi_t \sim \mathcal{N}(0, \mathbb{I}_d)$. Generate samples starting with $X_T \sim p_{\text{ref}} \approx \mathcal{N}(0, \mathbb{I})$ and iteratively denoising it: $X_t = (2-\alpha)X_{t+1} + (1-\alpha^2)s_{\theta^*}(t+1, X_{t+1}) + \sqrt{1-\alpha^2}\xi_{t+1}$
- U-net architecture: DSM implemented with encoder–decoder (self-attention) + skip connections (ResNet). Time t embedded with sinusoidal positinotial encodings and injected into residual blocks (spatial addition or adaptive group normalization layers). Exponential Moving Average of parameters for stability: $\bar{\theta}_{n+1} = (1-\beta)\bar{\theta}_n + \beta\theta_n$.
- Conditional diffusion: condition DSM on additional input (label/prompt Y) to learn $s_\theta(t, X_t; Y)$, embedded like time embeddings in U-net; objective becomes $\theta^* = \arg\min_\theta \frac{1}{2} \sum_{t=1}^T \mathbb{E}_{X_0, Y} \mathbb{E}_{X_t|X_0}[\|s_\theta(t, X_t^Y; Y) - \nabla \log p(X_t^Y \mid X_0^Y)\|^2]$

Lab Exercises

- Model not identifiable bcs stays the same if you switch the labels assigned to clusters
- $f(z) = \sum_{i=0}^d a_i z^i$ pos. coeffs, k_1 kernel, $k(x, x') = \sum_{i=0}^d a_i (k_1(x, x'))^i$ is a kernel
- Multiply Softmax by $exp(\max_j(x_j))/exp(\max_j(x_j))$ for numerical stability
- $H = -\sum_{i=1}^d y_i \ln(z_i)$; $\frac{\partial H}{\partial x_j} = \sum_{k=1}^d \frac{\partial H}{\partial z_k} \frac{\partial z_k}{\partial x_j}$; $\frac{\partial H}{\partial x_j} = \frac{\partial H}{\partial z_j} \frac{\partial z_j}{\partial x_j} + \sum_{k\neq j} \frac{\partial H}{\partial z_k} \frac{\partial z_k}{\partial x_j}$
- Minimize dot product between w, x; x must point in opposite direction of w, $w/\|w\|_2$
- $\nabla(\mathbf{x}^\top \mathbf{A}\mathbf{x}) = (\mathbf{A} + \mathbf{A}^\top)\mathbf{x}$, if $\mathbf{A}$ was symmetric ($\mathbf{A} = \mathbf{A}^\top$), it would just be $2\mathbf{A}\mathbf{x}$